

# ADAPTIVE TREE BUDGETS AND EXACT TREE-BLOCK VERIFICATION FOR BLOCK-DIFFUSION SPECULATIVE DECODING

**Anonymous authors**

Paper under double-blind review

## ABSTRACT

Block-diffusion drafters expose many plausible future tokens in one forward pass, but a decoding system must decide both *which* candidate prefixes deserve target computation and *how* to sample from the verified tree without changing the target distribution. We present ADAPTIVETREE-TBV, a protocol-separated system with two components. At temperature zero, ADAPTIVETREE selects a node budget online from nested DDTree prefixes using an acceptance-calibrated, budget-aware latency model. At positive temperature, TBV keeps the official DDTree proposal tree and every target probability row fixed, but replaces all-row categorical sampling and host traversal by a single persistent CUDA block that performs inverse-CDF sampling only on reached rows. We prove that the tree forward produces path-local target conditionals, derive the complete variable-length block law, establish exact target-distribution preservation under history-adaptive trees, and prove equivalence of the parallel chunk scan to global inverse-CDF sampling. The same-tree result also implies that TBV cannot improve acceptance length over DDTree; any speed difference must come from execution. We specify a fail-closed evaluation on Qwen3-4B and Qwen3-8B across reasoning, code, and dialogue tasks. Quantitative result fields are intentionally blank pending a fresh, audited run.

## 1 INTRODUCTION

Autoregressive language models sample one token per target-model step. Speculative decoding amortizes this sequential cost by proposing several future tokens with a cheaper model and verifying them in parallel while preserving the target law (Leviathan et al., 2023; Chen et al., 2023). Its practical speed is governed by three coupled quantities: draft latency, target verification cost, and the number of tokens committed per verification round.

Block diffusion is an appealing drafter because it predicts all future positions in one pass. DFLASH conditions a lightweight block-diffusion model on target features and verifies one path (Chen et al., 2026). DDTree reuses the same position-wise distributions to construct a high-mass prefix tree, then obtains target logits for all tree nodes with one ancestor-masked forward pass (Ringel & Romano, 2026). A larger tree can cover more target continuations, but it also increases tree construction, masked attention, vocabulary projection, sampling, and cache-compaction costs. Therefore, neither a globally fixed budget nor acceptance length alone determines end-to-end throughput.

This work separates two questions that are easy to conflate. First, at deterministic decoding temperature  $T = 0$ , how should the tree-node budget adapt to the prompt, context length, and measured device costs? Second, at stochastic temperature  $T > 0$ , how can the already verified tree be traversed with less control overhead while retaining exactly the same conditional target law? Figure 1 shows the resulting architecture. The two protocols share the block-diffusion proposal and tree representation, but they are evaluated separately: ADAPTIVETREE is currently a greedy decoder, whereas TBV is the stochastic verifier.

Our contributions are:

Common proposal and target computation

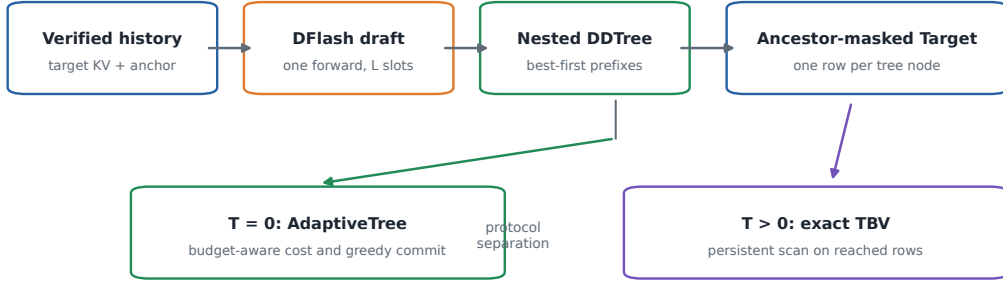


Figure 1: System overview. A single block-diffusion forward produces  $L$  position-wise distributions. At  $T = 0$ , ADAPTIVETREE chooses a nested tree budget using a corrected per-budget cost model. At  $T > 0$ , TBV and DDTree receive byte-identical tree topology and FP64 target probabilities; only the post-forward verifier changes.

- a corrected latency-aware tree controller that treats tree construction as a budget-dependent cost, expands the candidate range to 256 nodes, and disables periodic exploration after a one-pass warm-up;
- an exact same-tree verifier that uses a persistent CUDA block and a parallel chunk scan to sample only target rows reached by ancestral traversal;
- a strict derivation of the emitted block distribution, including adaptive-tree conditioning, multi-round exactness, expected accepted length, tree-attention correctness, and the inverse-CDF kernel; and
- a preregistered, failure-closed evaluation in which all model revisions, inputs, backends, temperatures, method positions, scorers, and same-tree witnesses are audited before any speed claim is released.

## 2 RELATED WORK

**Speculative decoding.** Independent formulations of speculative decoding established exact distribution preservation for a draft-and-correct pipeline (Leviathan et al., 2023; Chen et al., 2023). Medusa (Cai et al., 2024) and EAGLE-family systems (Li et al., 2024) use multiple heads or feature-space drafting to expose tree-shaped candidates. Block Verification jointly couples a single draft chain to target rows and proves an optimal expected emission length for its setting (Sun et al., 2024). Our TBV problem is different: the proposal tree and target rows are frozen to DDTree, and the accepted-prefix law is unchanged.

**Adaptive trees and block diffusion.** OPT-Tree optimizes draft-tree structure under a node budget (Wang et al., 2025). DDTree adapts best-first tree construction to position-wise distributions emitted by one block-diffusion forward (Ringel & Romano, 2026), building on DFLASH (Chen et al., 2026). We do not claim the best-first tree or adaptive tree construction as new. Our deterministic component instead selects among nested DDTree budgets using measured device latency, with corrected attribution of the node-dependent construction time.

**GPU execution.** GPU scan and reduction primitives make it possible to express a vocabulary-wide categorical draw without a sequence of host decisions. The proposed verifier keeps one CUDA block resident across tree depths, performs a chunked FP64 reduction and exclusive scan on the current row, and traverses the sparse tree on device. Unlike methods that alter the proposal-target coupling, this is an execution substitution for the same ancestral target sampler.

### 3 PROBLEM SETUP AND ARCHITECTURE

Let  $h \in \mathcal{V}^*$  be the verified history and let the target model define

$$p_h(x; T) = \frac{\exp(z_\theta(x | h)/T)}{\sum_{a \in \mathcal{V}} \exp(z_\theta(a | h)/T)}, \quad T > 0. \quad (1)$$

All stochastic results below hold for every fixed finite  $T > 0$ ;  $T = 1$  is only the registered experimental instance. At  $T = 0$ ,  $p_h$  is instead replaced by the implementation’s deterministic argmax with a fixed tie rule. One DFLASH forward produces logits for  $L$  future slots. For tree construction, they induce position-wise proposal distributions  $q_d$  and prefix weights

$$Q(u_1 \dots u_k) = \prod_{d=1}^k q_d(u_d), \quad 1 \leq k \leq L. \quad (2)$$

This factorization is a proposal surrogate; it is not assumed to equal the target’s autoregressive joint distribution.

#### 3.1 PROBABILITY TREES AND A SINGLE TARGET FORWARD

A finite probability tree is a prefix-closed rooted tree  $\mathcal{T} = (V, E)$  with root  $\rho$ . Every non-root node  $v$  has parent  $\text{par}(v)$  and edge label  $a(v) \in \mathcal{V}$ ; labels are unique among siblings. The token prefix along the root-to- $v$  path is  $s(v)$  and its depth is  $|s(v)|$ . The candidate set at  $v$  is  $\mathcal{C}(v) = \{a(w) : \text{par}(w) = v\}$ .

The root represents history  $h$ . All non-root tokens are flattened into one sequence, assigned their path-consistent absolute positions, and evaluated with the mask

$$M_{uv} = \begin{cases} 0, & u \text{ is in the old prefix or } u \preceq v, \\ -\infty, & \text{otherwise,} \end{cases} \quad (3)$$

where  $u \preceq v$  means that  $u$  is an ancestor of  $v$ . Thus each row attends to the old prefix and exactly its own ancestors, never to a sibling. Under the standard path-local transformer assumption stated in Appendix B, the row at node  $v$  equals the sequential conditional

$$p_v(x) \equiv p_\theta(x | h \| s(v); T). \quad (4)$$

#### 3.2 CORRECTED ADAPTIVETREE AT $T = 0$

The DDTREE best-first heap enumerates non-root nodes in non-increasing  $Q(u)$ . Its first  $b$  nodes form a valid nested tree  $\mathcal{T}_b$ . The proposal-mass proxy

$$M_b = \sum_{u \in \mathcal{T}_b} Q(u) \quad (5)$$

equals the expected covered prefix length under the factorized draft surrogate (Appendix D). ADAPTIVETREE enumerates once to  $B_{\max} = 256$  and selects from

$$\mathcal{B} = \{30, 45, 60, 80, 100, 128, 160, 192, 256\}. \quad (6)$$

After one warm-up observation per budget, it chooses

$$b_t \in \arg \max_{b \in \mathcal{B}} \frac{1 + \min\{L, \hat{s}_t M_b\}}{\hat{D}_t + \hat{G}_t(b) + \hat{V}_t(b) + \hat{K}_t(b)}, \quad (7)$$

with smaller  $b$  breaking ties. Here  $D$  is block-draft time,  $G$  is tree construction,  $V$  includes tree compilation and target verification, and  $K$  is commit/cache time. All estimates are exponential moving averages with coefficient 0.2, and  $\hat{s}_t \in [0, 2]$  calibrates proposal mass to observed accepted draft tokens. The correction relative to the historical controller is the placement of  $G_t(b)$  in the budget-specific denominator: constructing more nodes is not a fixed cost. The canonical method performs no scheduled exploration after warm-up. Budget choice changes efficiency but not the greedy acceptance rule; Appendix E states the deterministic equivalence result.

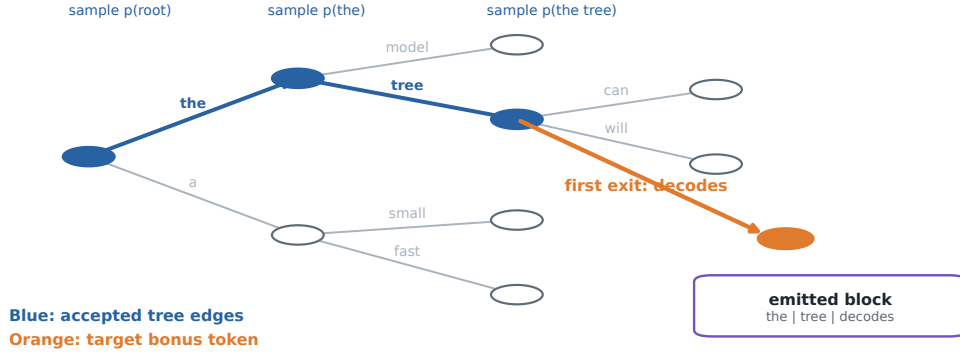


Figure 2: One TBV round. Blue edges are sampled target tokens that match verified children; the orange edge is the first sampled token outside the tree and becomes the bonus. The emitted block is the accepted root-to-node prefix followed by that bonus.

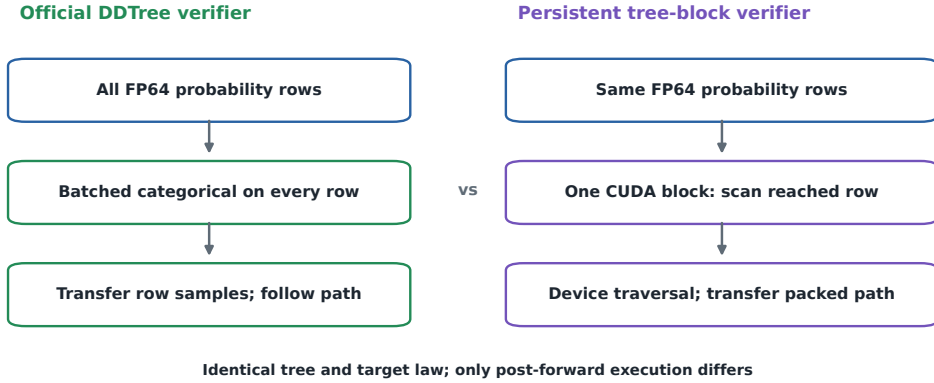


Figure 3: Post-forward execution. Official DDTREE samples all  $|V|$  probability rows in one batched categorical and follows the path after a host transfer. TBV precomputes the same FP64 rows, then one persistent CUDA block scans only reached rows and returns one packed path. Acceptance is identical in distribution on a frozen tree.

### 3.3 TREE-BLOCK VERIFICATION AT $T > 0$

TBV receives the same  $\mathcal{T}$  and  $\{p_v\}_{v \in V}$  as DDTREE. It draws independent uniforms  $U_0, \dots, U_D \sim \text{Unif}[0, 1)$ , with  $D$  the tree depth. Starting at  $v_0 = \rho$ , it samples

$$X_d = F_{v_d}^{-1}(U_d), \quad F_v^{-1}(u) = \min \left\{ x : \sum_{a \leq x} p_v(a) > u \sum_a p_v(a) \right\}. \quad (8)$$

If  $X_d \in \mathcal{C}(v_d)$ , the unique matching child is accepted and traversal continues. Otherwise,  $X_d$  is the bonus token and the round ends. A leaf necessarily ends the round. Figure 2 illustrates this first-exit view.

## 4 EXACTNESS OF TREE-BLOCK VERIFICATION

We give the core derivation here and a filtration-level proof in Appendix A. For  $v \in V$ , define the target probability of its tree prefix

$$\pi(v) = \prod_{(u \rightarrow w) \in \text{path}(\rho, v)} p_u(a(w)), \quad \pi(\rho) = 1, \quad (9)$$

and its exit mass

$$e(v) = \sum_{x \notin \mathcal{C}(v)} p_v(x) = 1 - \sum_{x \in \mathcal{C}(v)} p_v(x). \quad (10)$$

The registered ancestral kernel never needs to materialize  $e(v)$ . When terminal-mass checks are used for validation, uncovered probability is summed directly rather than obtained by subtraction, which could erase a small positive tail; the final equality in (10) is a real-arithmetic identity.

**Theorem 1** (Complete block law for arbitrary  $T > 0$ ). *Fix  $h$ , a valid finite tree  $\mathcal{T}$ , and exact path-local rows  $p_v$ . For every node  $v$  and token  $x \notin \mathcal{C}(v)$ , the probability that TBV emits  $s(v)||x$  is*

$$\mathbb{P}\{Z = s(v)||x \mid h, \mathcal{T}\} = \pi(v)p_v(x). \quad (11)$$

*The events in (11) are disjoint and sum to one.*

*Proof.* The event requires sampling each edge label on the unique path to  $v$ , followed by  $x$  at  $v$ . The uniforms are independent, and inverse-CDF sampling returns row  $p_u$ , so multiplying the conditional factors gives  $\pi(v)p_v(x)$ . Sibling-label uniqueness makes the path unique. Every target continuation has a unique first token that is not a child of its current tree node; a finite tree guarantees this occurs no later than a leaf. Hence these first-exit events partition the target sample space. Equivalently,

$$\sum_{v \in V} \pi(v)e(v) = 1, \quad (12)$$

which follows by recursively splitting each prefix mass into child-prefix masses and exit mass.  $\square$

**Theorem 2** (Adaptive, multi-round target-law preservation for arbitrary  $T > 0$ ). *Fix any finite temperature  $T > 0$ . At round  $t$ , let the tree  $\mathcal{T}_t$  be any finite, valid, history-adaptive random tree measurable with respect to the verified history, draft randomness, and prior runtime state, but independent of the fresh verifier uniforms conditional on that state. If tree rows satisfy (4), concatenating TBV blocks has exactly the temperature- $T$  target autoregressive distribution.*

*Proof.* Condition on the pre-round state and  $\mathcal{T}_t$ . Couple TBV to a serial target sampler by using the same  $U_d$  at each visited context. Both samplers emit the same token from  $p_\theta(\cdot \mid h||X_{<d}; T)$ . The tree only determines the stopping time at which the current verified block ends; it never changes a sampled token. The bonus is already the next serial target token, and the following round resumes from the identical history. Induction over within-round depth and then over rounds gives equality of every finite-dimensional token distribution. Averaging over the conditioned state and random tree preserves the equality.  $\square$

**Corollary 1** (Same-tree acceptance identity). *Let  $A$  be the number of accepted tree edges in a round. Then*

$$\mathbb{E}[A \mid h, \mathcal{T}] = \sum_{v \in V \setminus \{\rho\}} \pi(v). \quad (13)$$

*Consequently, TBV and ancestral DDTree have identical accepted-length distributions on the same tree and target rows.*

*Proof.* For each non-root node  $v$ , the event that traversal reaches  $v$  has probability  $\pi(v)$ . Summing its indicator over all non-root nodes and applying linearity of expectation yields (13). The two implementations use the same row-wise target draws and child transition rule, so their complete first-exit laws agree by Theorem 1.  $\square$

The corollary restricts the performance claim: a same-tree verifier cannot be faster because it accepts more tokens. It can only reduce the time after target logits exist. Figure 3 makes that controlled difference explicit.

Table 1: Protocol separation. Cross-protocol averaging is forbidden.

| Protocol | Methods                                                                                                       | Frozen conditions                                                                                                           |
|----------|---------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| $T = 0$  | Target, official DFLASH, DDTREE<br>$B \in \{16, 32, 64, 128, 256, 512, 1024\}$ ,<br>ADAPTIVETREE and controls | Qwen3-4B/8B; 10 datasets; BF16; $L = 15$ ;<br>Target SDPA main comparison; Draft FlashAt-<br>tention 2; 2,048-token cap     |
| $T = 1$  | Target, official DFLASH, DDTREE $B =$<br>45, TBV $B = 45$                                                     | Qwen3-4B/8B; 8 datasets; seeds 17/29/43; Tar-<br>get/Draft SDPA; FP64 sampling probabilities;<br>$L = 15$ ; 2,048-token cap |

## 5 PERSISTENT INVERSE-CDF KERNEL

Let a reached row contain nonnegative weights  $w_0, \dots, w_{m-1}$  with total  $W > 0$ . The kernel partitions the vocabulary contiguously across  $P = 640$  threads. Thread  $j$  accumulates  $S_j = \sum_{x \in I_j} w_x$ . A block-wide exclusive scan yields  $A_j = \sum_{i < j} S_i$  and  $W = \sum_j S_j$ . For threshold  $R = UW$ , exactly one nonempty chunk satisfies

$$A_j \leq R < A_j + S_j. \quad (14)$$

That thread scans within  $I_j$  to find the first token whose local cumulative mass crosses  $R - A_j$ . This is exactly the global inverse CDF in (8); the formal statement, including zero-mass chunks and boundary events, is in Appendix C.

After selecting token  $x$ , thread zero looks up the edge  $(v, x)$ . If present, the kernel updates the current node and repeats; otherwise it writes the accepted path length and bonus token. One launch therefore covers all visited depths. The registered method intentionally retains the common full-row pipeline: BF16 target logits are converted to FP64 probability rows before either verifier. More aggressive direct-logit and lazy-projection variants are implementation ablations, not the main method, because they change the work boundary and did not pass the frozen selection gate.

With  $A$  accepted nodes, vocabulary size  $m$ , and tree budget  $B$ , the persistent kernel performs  $A + 1$  row scans, each  $O(m/P)$  work per thread plus a  $P$ -way scan, and an  $O(B)$  edge lookup by one thread per depth. It stores  $O(P + D)$  shared/control state and reads no unused probability row. The official batched sampler is highly optimized, so this complexity reduction is a hardware hypothesis rather than a theorem of wall-clock speedup.

## 6 EXPERIMENTAL DESIGN

The experiment is intentionally split by sampling law. Table 1 records the frozen scope; Figure 4 visualizes coverage. Results are populated only after every integrity and fairness gate passes.

**Datasets and models.** The target/draft pairs are Qwen3-4B/Qwen3-4B-DFlash-b16 and Qwen3-8B/Qwen3-8B-DFlash-b16 (Qwen Team, 2025), each pinned to a 40-character revision. The  $T = 0$  matrix contains GSM8K (Cobbe et al., 2021), MATH-500, AIME 2024/2025, HumanEval (Chen et al., 2021), MBPP-sanitized, LiveCodeBench (Jain et al., 2024), SWE-bench Lite (Jimenez et al., 2024), MT-Bench (Zheng et al., 2023), and Alpaca. The  $T = 1$  scored matrix excludes SWE-bench and Alpaca until their execution or external-judge contracts are registered. Sampling and maximum counts follow the pinned DDTREE protocol.

**Metrics.** We report decode throughput, time per output token, speedup relative to the same-pair Target baseline, accepted draft tokens per target verification, generated length, peak memory, and task quality. For  $T = 1$ , every method is paired by model, dataset, source, and seed; 95% intervals use 10,000 source-clustered bootstrap samples with all seeds for one source kept in the same cluster. For  $T = 0$ , the official single deterministic pass receives descriptive point estimates only. We do not compare proposal-normalized acceptance ratios between DFLASH paths and DDTREE nodes because their denominators differ.

**Fairness gates.** All methods share prompts, stop rules, output caps, model revisions, and a balanced cyclic method order within each dataset. At  $T = 1$ , DDTREE and TBV must emit a runtime

|          | Target | DFlash | DDTree | AdaptiveTree | TBV |
|----------|--------|--------|--------|--------------|-----|
| 4B / T=0 |        |        |        |              | n/a |
| 8B / T=0 |        |        |        |              | n/a |
| 4B / T=1 |        |        |        | n/a          |     |
| 8B / T=1 |        |        |        | n/a          |     |

Blank underline = reserved result field after audit

Figure 4: Preregistered matrix. Empty cells are intentional result slots, not missing experimental conditions. ADAPTIVETREE is evaluated only at  $T = 0$ ; TBV only at  $T = 1$ .

Table 2:  $T = 0$  primary architecture comparison. Speedup is relative to the paired Target-only baseline. Every result cell is intentionally blank.

| Model    | Method            | Speedup | TPOT (ms) | Accept/round | Exact output (%) |
|----------|-------------------|---------|-----------|--------------|------------------|
| Qwen3-4B | Target            | —       | —         | —            | —                |
|          | DFLASH            | —       | —         | —            | —                |
|          | best fixed DDTree | —       | —         | —            | —                |
|          | ADAPTIVETREE      | —       | —         | —            | —                |
| Qwen3-8B | Target            | —       | —         | —            | —                |
|          | DFLASH            | —       | —         | —            | —                |
|          | best fixed DDTree | —       | —         | —            | —                |
|          | ADAPTIVETREE      | —       | —         | —            | —                |

witness showing identical parent arrays, edge tokens, complete FP64 probability-tensor hashes, generator-state hashes, and distinct executed callable hashes. Target and Draft attention backends are identical within the  $T = 1$  comparison. Objective scorers run in an identity-pinned sandbox. A speed superiority claim is permitted only if the full 4B/8B matrix is complete and the paired 95% confidence-interval lower bound exceeds one.

## 7 RESULTS

**Current status.** No numerical conclusion is made in this draft. In particular, the blank cells must not be filled from pilot runs, incomplete submatrices, microbenchmarks, or runs that fail the output-quality, same-tree, environment, order-balance, coverage, or report-reconstruction gates.

## 8 DISCUSSION AND LIMITATIONS

The system deliberately makes a narrower claim than “tree decoding is faster.” ADAPTIVETREE optimizes a measured throughput surrogate at  $T = 0$ ; it is neither reinforcement learning nor a proof of globally optimal latency. TBV is exact for the stochastic target law, but its same-tree accepted length is provably identical to DDTree. The persistent kernel may lose to a tuned batched categorical on a particular GPU, vocabulary size, or tree shape. Such a result is a valid negative outcome, not a correctness failure.

The mathematical theorems use exact real arithmetic and path-local target rows. The implementation uses BF16 model computation and FP64 probability arithmetic. Floating-point reductions, finite



Table 3:  $T = 1$  same-tree comparison. Parent arrays, edge tokens, and FP64 target probability rows are required to match before these fields can be filled. CI denotes the paired source-clustered 95% interval.

| Model    | Method | Speedup vs Target | Speedup vs DDTREE | 95% CI | Accept/round |
|----------|--------|-------------------|-------------------|--------|--------------|
| Qwen3-4B | DFLASH | —                 | —                 | —      | —            |
|          | DDTREE | —                 | —                 | —      | —            |
|          | TBV    | —                 | —                 | —      | —            |
| Qwen3-8B | DFLASH | —                 | —                 | —      | —            |
|          | DDTREE | —                 | —                 | —      | —            |
|          | TBV    | —                 | —                 | —      | —            |

mask sentinels, and shape-dependent attention kernels can prevent bitwise equality even when the abstract sampling law is exact. We therefore separate distributional unit tests from real-checkpoint output audits and report any greedy BF16 mismatch rather than filtering it out. Finally, results on two Qwen3 model sizes do not establish behavior for larger dense or mixture-of-experts models.

## REPRODUCIBILITY STATEMENT

The full algorithm, assumptions, and proofs appear in Sections 3–5 and Appendices A–E. Appendix F lists the frozen model revisions, dataset counts, seeds, backends, precision settings, method-order policy, and audit gates. The supplementary code contains the official-source hashes, CUDA implementation, exact finite-law tests, same-tree runtime witness, run manifests, and report rebuilders. Result tables remain blank until a fresh run satisfies the registered completion checks.

## ETHICS STATEMENT

This work studies inference efficiency and does not train or release a new language model. Faster generation can reduce compute cost but may also increase the scale at which model outputs are produced. Benchmark prompts and model outputs should be handled under their source licenses and privacy terms. We report quality alongside speed and do not treat faster generation as evidence of safer or more reliable model behavior.

## REFERENCES

- Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.
- Jian Chen, Yesheng Liang, and Zhijian Liu. DFlash: Block diffusion for flash speculative decoding. *arXiv preprint arXiv:2602.06036*, 2026.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Rei Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.



- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, pp. 19274–19286, 2023.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-2: Faster inference of language models with dynamic draft trees. *arXiv preprint arXiv:2406.16858*, 2024.
- Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Liran Ringel and Yaniv Romano. Accelerating speculative decoding with block diffusion draft trees. *arXiv preprint arXiv:2604.12989*, 2026.
- Ziteng Sun, Uri Mendlovic, Yaniv Leviathan, Asaf Aharoni, Ahmad Beirami, Jae Hun Ro, and Ananda Theertha Suresh. Block verification accelerates speculative decoding. *arXiv preprint arXiv:2403.10444*, 2024.
- Jikai Wang, Yi Su, Juntao Li, Qingrong Xia, Zi Ye, Xinyu Duan, Zhefeng Wang, and Min Zhang. OPT-tree: Speculative decoding with adaptive draft tree structure. *Transactions of the Association for Computational Linguistics*, 13:188–199, 2025. doi: 10.1162/tacl.a\_00735.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, et al. Judging LLM-as-a-judge with MT-bench and chatbot arena. *Advances in Neural Information Processing Systems*, 36, 2023.

## AI USE STATEMENT

Generative AI tools were used to assist with source-code navigation, LaTeX drafting, figure scripting, and language editing. The authors are responsible for the mathematical claims, implementation-to-equation correspondence, citations, experiment execution, and final manuscript. No AI-generated numerical result is reported; all quantitative fields are intentionally blank pending audited experiments.

## A COMPLETE PROBABILITY MODEL

### A.1 FILTRATION AND ADMISSIBLE ADAPTIVE TREES

Let  $Y_1, Y_2, \dots$  denote generated tokens and let  $\mathcal{F}_t$  contain the prompt, all tokens committed before round  $t$ , draft-model randomness, previous verifier randomness, caches, timing observations, and controller state. The round- $t$  tree  $\mathcal{T}_t$  is *admissible* when it is finite, prefix-closed, has unique sibling labels, and is measurable with respect to  $\mathcal{F}_t$  plus fresh draft randomness generated before verification. The verifier uniforms  $U_{t,0}, \dots, U_{t,D_t}$  are conditionally independent of  $\mathcal{T}_t$  given that pre-verification state.

This condition allows the tree to depend on the full history, proposal logits, prompt class, past acceptances, past timings, and hardware state. It forbids choosing or expanding the current tree after observing a fresh target sample from the row being verified. The target rows themselves are deterministic functions of the model, history, and tree paths.

### A.2 PATH AND TERMINAL-EVENT DERIVATION

Write  $v_0 = \rho, v_1, \dots, v_k = v$  for the path to  $v$ , and  $a_i = a(v_i)$ . The event that traversal reaches  $v$  is

$$R_v = \{X_0 = a_1, \dots, X_{k-1} = a_k\}. \quad (15)$$

Conditional on the tree and history,

$$\mathbb{P}(R_v \mid \mathcal{F}_t, \mathcal{T}_t) = \prod_{i=0}^{k-1} \mathbb{P}(X_i = a_{i+1} \mid X_{<i} = a_{1:i}, \mathcal{F}_t, \mathcal{T}_t) \quad (16)$$

$$= \prod_{i=0}^{k-1} p_{v_i}(a_{i+1}) = \pi(v). \quad (17)$$

For  $x \notin \mathcal{C}(v)$ , define  $E_{v,x} = R_v \cap \{X_k = x\}$ . Then

$$\mathbb{P}(E_{v,x} \mid \mathcal{F}_t, \mathcal{T}_t) = \pi(v)p_v(x). \quad (18)$$

The events  $E_{v,x}$  are pairwise disjoint: two distinct outputs either diverge on a sampled token or stop at different first-exit positions. They are exhaustive because every root-to-leaf walk reaches a node without a matching child in finite depth. Summing (18) yields one.

An algebraic proof uses the recursion

$$\pi(v) = \pi(v)e(v) + \sum_{w:\text{par}(w)=v} \pi(w). \quad (19)$$

Applying it from the root down telescopes all internal child masses and leaves  $\sum_v \pi(v)e(v) = \pi(\rho) = 1$ . Conditional on exiting at  $v$ , the bonus token law is

$$\mathbb{P}(X = x \mid \text{exit at } v) = \frac{p_v(x)\mathbf{1}\{x \notin \mathcal{C}(v)\}}{e(v)} \quad (20)$$

when  $e(v) > 0$ . Multiplying by terminal-node mass  $\pi(v)e(v)$  recovers (18). This terminal-mass factorization is a proof device; the registered kernel implements the equivalent ancestral process.

### A.3 STOPPING-TIME PROOF OF AUTOREGRESSIVE EXACTNESS

Consider an infinite serial target sequence generated from independent uniforms  $U_1, U_2, \dots$  via inverse CDF at the current history. Given an admissible tree, define

$$\tau = \min\{k \geq 1 : Y_k \notin \mathcal{C}(v_{k-1})\}, \quad (21)$$

where  $v_{k-1}$  is the tree node reached by  $Y_{1:k-1}$ . Because  $\mathcal{T}$  is finite,  $\tau \leq D + 1$ . The event  $\{\tau = k\}$  is determined by  $Y_{1:k}$  and the fixed tree, so  $\tau$  is a bounded stopping time. TBV emits exactly  $Y_{1:\tau}$ . Restarting with fresh uniforms after  $\tau$  is distributionally identical to continuing the serial sampler, because the target conditional depends only on the committed history. Repeating at successive bounded stopping times partitions, but never changes, the serial target sequence. This proves Theorem 2.

#### A.4 ACCEPTED-LENGTH DISTRIBUTION

For  $d \geq 1$ , let  $V_d$  be the set of depth- $d$  nodes. Since these path events are mutually exclusive,

$$\mathbb{P}(A \geq d \mid h, \mathcal{T}) = \sum_{v \in V_d} \pi(v). \quad (22)$$

The tail-sum identity gives

$$\mathbb{E}[A \mid h, \mathcal{T}] = \sum_{d=1}^D \mathbb{P}(A \geq d \mid h, \mathcal{T}) \quad (23)$$

$$= \sum_{v \in V \setminus \{\rho\}} \pi(v). \quad (24)$$

The entire distribution of  $A$ , not merely its expectation, is a function only of  $\mathcal{T}$  and target rows. Therefore an implementation-only verifier substitution cannot honestly claim a systematic acceptance improvement under a same-tree replay.

## B CORRECTNESS OF THE TREE TARGET FORWARD

**Assumption 1** (Path locality). *The target is in deterministic evaluation mode. For every tree node, token identity, absolute position, positional transformation, and allowed attention keys equal those of its serial root-to-node sequence. Non-attention operations act independently per position. Masked attention probabilities are exactly zero, and the incoming KV cache equals the serial cache for the verified history.*

**Lemma 1** (Tree-row equivalence). *Under path locality, the hidden state, KV entries, and output logits at node  $v$  equal those obtained by serially evaluating  $h \parallel s(v)$ .*

*Proof.* Proceed by transformer layer. At the input, the token embedding and positional representation of  $v$  match serial evaluation. Suppose all representations and cached keys/values visible to  $v$  match at layer  $\ell$ . Equation (3) exposes exactly the old prefix and ancestors with identical positions, so queries, keys, values, attention normalization, and weighted sums match. Sibling representations are masked and contribute zero. Position-wise residual, normalization, and feed-forward operations therefore match, establishing the induction step. The final language-model head gives the same logits, and the layer-wise keys and values match as well.  $\square$

In finite precision, a large negative mask sentinel is not literally  $-\infty$ , and different attention shapes may select different kernels. The lemma is the exact-computation contract; real-checkpoint audits separately test token or distributional behavior.

## C EQUIVALENCE OF THE PARALLEL CHUNK SCAN

Let contiguous chunks  $I_0, \dots, I_{P-1}$  partition  $\{0, \dots, m-1\}$ . Define

$$S_j = \sum_{x \in I_j} w_x, \quad A_j = \sum_{i < j} S_i, \quad C(x) = \sum_{a=0}^x w_a. \quad (25)$$

**Theorem 3** (Chunk-scan inverse-CDF equivalence). *For  $w_x \geq 0$ ,  $W = \sum_x w_x > 0$ , and  $U \sim \text{Unif}[0, 1)$ , the two-stage rule*

1. *choose the unique chunk  $j$  satisfying  $A_j \leq UW < A_j + S_j$ ;*
2. *return the smallest  $x \in I_j$  satisfying  $C(x) > UW$*

*returns token  $x$  with probability  $w_x/W$  and agrees almost surely with a global inverse-CDF draw.*

*Proof.* Zero-mass chunks contain no threshold interval and cannot be selected. For every positive-mass chunk,  $[A_j, A_j + S_j)$  is exactly the disjoint interval covered by its tokens. Within the selected chunk, subtracting the exclusive prefix  $A_j$  leaves the same ordered cumulative token intervals as the global CDF. Thus token  $x$  is returned exactly when

$$C(x - 1) \leq UW < C(x), \quad (26)$$

with  $C(-1) = 0$ . This interval has length  $w_x$ , so its probability under uniform  $UW$  on  $[0, W)$  is  $w_x/W$ . Threshold equality at an internal boundary has probability zero for an ideal continuous uniform; the implementation fixes a strict comparison rule for reproducibility.  $\square$

The CUDA kernel applies this theorem at the current node, then performs a deterministic child lookup. Conditional on reaching the next node, the next independent uniform invokes the theorem again. Induction over depth yields the same path as serial inverse-CDF ancestral sampling for the supplied uniform vector.

## D ADAPTIVE BUDGET PROPERTIES AT $T = 0$

For a prefix-closed tree  $\mathcal{T}$  and an auxiliary draft sequence  $X_{1:L} \sim \prod_d q_d$ , define its covered prefix length

$$A_{\mathcal{T}}(X) = \max(\{0\} \cup \{d : X_{1:d} \in \mathcal{T}\}). \quad (27)$$

**Proposition 1** (Mass proxy identity). *For every finite prefix-closed  $\mathcal{T}$ ,*

$$\mathbb{E}_Q[A_{\mathcal{T}}(X)] = \sum_{u \in \mathcal{T}} Q(u). \quad (28)$$

*Proof.* Prefix closure implies  $\{A_{\mathcal{T}} \geq d\} = \{X_{1:d} \in \mathcal{T}\}$ . Using the tail-sum formula and disjoint depth- $d$  prefixes,

$$\mathbb{E}_Q[A_{\mathcal{T}}] = \sum_{d=1}^L \sum_{u \in \mathcal{T}: |u|=d} Q(u) = \sum_{u \in \mathcal{T}} Q(u). \quad (29)$$

$\square$

The best-first DDTREE predecessor rule guarantees that a node enters the heap only after its parent or preceding sibling. Since both parent and preceding sibling have no smaller weight, the heap frontier always contains a predecessor of every unenumerated candidate. Therefore the next pop is a maximum-weight remaining node, every enumeration prefix is prefix-closed, and  $\mathcal{T}_b$  maximizes the proxy mass among size- $b$  trees in the retained candidate space. This is a property of DDTREE, not a new optimality claim for the latency controller. Equation (7) uses this proxy only as one input; it does not imply global throughput optimality.

## E GREEDY OUTPUT EQUIVALENCE

At  $T = 0$ , let  $g(c)$  be the target argmax under the fixed tie rule. Starting from verified context  $c$ , the tree verifier queries the root row. If  $g(c)$  labels a child, it accepts that child and repeats at the corresponding path-local row. Otherwise it emits  $g(c)$  as bonus. By induction, every accepted node and the bonus equal the next tokens of serial greedy decoding. The selected budget may depend arbitrarily on the current proposal, prior timings, acceptance history, and controller state, because correctness holds for every valid tree. Cache compaction retains the root and accepted path KV entries but not the bonus, which becomes the next round’s anchor. A second induction over rounds proves equality to serial greedy decoding under the same EOS and length rules, subject to the exact path-locality assumption.

Table 4: Frozen model identities.

| Role            | Qwen3-4B pair          | Qwen3-8B pair          |
|-----------------|------------------------|------------------------|
| Target revision | 1cfa9a720891...b3df60c | b968826d9c46...de91218 |
| Draft revision  | b74e3a329c4d...2be72bd | 9b41424b7109...85333f4 |
| Weights         | BF16                   | BF16                   |
| Thinking        | disabled               | disabled               |

Table 5: Frozen dataset counts per model. MT-Bench contains two turns and therefore more generation calls than source rows.

|         | GSM8K | MATH | A24 | A25 | HE  | MBPP | LCB | SWE | MT | Alpaca |
|---------|-------|------|-----|-----|-----|------|-----|-----|----|--------|
| $T = 0$ | 128   | 128  | 30  | 30  | 164 | 128  | 128 | 128 | 80 | 128    |
| $T = 1$ | 128   | 128  | 30  | 30  | 164 | 128  | 128 | –   | 80 | –      |

## F FULL EXPERIMENTAL CONTRACT

The  $T = 0$  primary table compares all architectures under Target SDPA and Draft FlashAttention 2. A Target-FA2 Target/DFLASH run is auxiliary and is never silently mixed with tree methods. The method schedule is a deterministic cyclic rotation whose per-dataset position-count imbalance is at most one. The canonical ADAPTIVETREE candidate set is  $\{30, 45, 60, 80, 100, 128, 160, 192, 256\}$ , with a one-observation warm-up per budget, EWMA coefficient 0.2, and no periodic exploration.

At  $T = 1$ , both Target and Draft use SDPA,  $L = 15$ ,  $B = 45$ , and FP64 probabilities. DFLASH retains its official greedy draft proposal, while DDTREE and TBV use draft temperature 1. Seeds are 17, 29, and 43. Every completed record must bind the run identifier, model and data manifests, method order, source identifier, seed, prompt hash, output hash, timing, generated length, accepted length, and scorer result. The report is rebuilt from raw JSONL records rather than trusting cached tables.

## G ADDITIONAL BLANK RESULT TABLES

Table 6: Per-dataset  $T = 0$  speedup relative to Target.

| Model    | Method       | GSM8K | MATH | A24 | A25 | HE | MBPP | LCB | SWE | MT | Alpaca |
|----------|--------------|-------|------|-----|-----|----|------|-----|-----|----|--------|
| Qwen3-4B | DFLASH       | —     | —    | —   | —   | —  | —    | —   | —   | —  | —      |
|          | best DDTREE  | —     | —    | —   | —   | —  | —    | —   | —   | —  | —      |
|          | ADAPTIVETREE | —     | —    | —   | —   | —  | —    | —   | —   | —  | —      |
| Qwen3-8B | DFLASH       | —     | —    | —   | —   | —  | —    | —   | —   | —  | —      |
|          | best DDTREE  | —     | —    | —   | —   | —  | —    | —   | —   | —  | —      |
|          | ADAPTIVETREE | —     | —    | —   | —   | —  | —    | —   | —   | —  | —      |

Table 7: Per-dataset  $T = 1$  throughput speedup. The first value is versus Target; the parenthesized value is versus DDTREE.

| Model    | Method | GSM8K | MATH | A24 | A25 | HE | MBPP | LCB | MT |
|----------|--------|-------|------|-----|-----|----|------|-----|----|
| Qwen3-4B | DFLASH | —     | —    | —   | —   | —  | —    | —   | —  |
|          | DDTREE | —     | —    | —   | —   | —  | —    | —   | —  |
|          | TBV    | —     | —    | —   | —   | —  | —    | —   | —  |
| Qwen3-8B | DFLASH | —     | —    | —   | —   | —  | —    | —   | —  |
|          | DDTREE | —     | —    | —   | —   | —  | —    | —   | —  |
|          | TBV    | —     | —    | —   | —   | —  | —    | —   | —  |

Table 8: AdaptiveTree ablations at  $T = 0$ . Values are geometric-mean speedups relative to the paired Target baseline.

| Variant                   | Qwen3-4B | Qwen3-8B | Single changed factor                            |
|---------------------------|----------|----------|--------------------------------------------------|
| ADAPTIVETREE              | —        | —        | canonical controller                             |
| $B \leq 128$              | —        | —        | remove 160/192/256 candidates                    |
| legacy cost attribution   | —        | —        | treat tree build as fixed proposal cost          |
| with periodic exploration | —        | —        | restore one forced probe every 64 decisions      |
| no acceptance calibration | —        | —        | fix $\hat{s}_t = 1$                              |
| no measured latency       | —        | —        | rank budgets by calibrated proposal mass only    |
| frozen after warm-up      | —        | —        | disable subsequent EWMA updates                  |
| historical controller     | —        | —        | old budget, attribution, and exploration package |

Table 9: Verifier-only same-state replay. Latency ratios are DDTREE verifier time divided by candidate verifier time.

| Model    | $T = 1$ ratio | Kernel time ( $\mu s$ ) | Peak memory (MiB) | Law test |
|----------|---------------|-------------------------|-------------------|----------|
| Qwen3-4B | —             | —                       | —                 | —        |
| Qwen3-8B | —             | —                       | —                 | —        |

Table 10: Task-quality audit at  $T = 1$ . MT-Bench requires a separately registered external judge and is shown as not scored here.

| Model    | Method | Math accuracy | Code pass rate | Mean gen. length | MT-Bench |
|----------|--------|---------------|----------------|------------------|----------|
| Qwen3-4B | Target | —             | —              | —                | —        |
|          | DFLASH | —             | —              | —                | —        |
|          | DDTREE | —             | —              | —                | —        |
|          | TBV    | —             | —              | —                | —        |
| Qwen3-8B | Target | —             | —              | —                | —        |
|          | DFLASH | —             | —              | —                | —        |
|          | DDTREE | —             | —              | —                | —        |
|          | TBV    | —             | —              | —                | —        |